

Introduction to the Enterprise Control Language



Part 1

Concepts and Queries



Welcome! – Class Overview

- ✓ Introduction
- ✓ HPCC Overview
- ✓ Spraying Data
- ✓ ECL Language Basics Overview
- ✓ Creating ECL Definitions (hands-on)
- ✓ ECL Functions used in Basic Exercises
- ✓ Basic Query Exercises
- ✓ More ECL Functions, More Exercises
- ✓ Using SORT and DEDUP

Introduction



- ✓ Name
- ✓ How many years at LexisNexis?
- ✓ Favorite Programming Language (if any)?
- ✓ What are *your* plans with HPCC and ECL?
- ✓ Favorite hobby after hours

What are the primary goals of this class?

- ✓ Familiarity with ECL (Enterprise Control Language) Programming tools
 - ✓ ECL IDE
 - ✓ ECL Watch
- ✓ Understanding ECL Concepts and Syntax
- ✓ Mastery of five ECL fundamentals:
 - ✓ Defining and recognizing the **four** fundamental types of ECL definitions:
 - ✓ Boolean - TRUE/FALSE expressions
 - ✓ Value - Scalar value expressions
 - ✓ Set - Sets of scalar values
 - ✓ Record Set - Sets of data records
 - ✓ Record Set Filtering - “WHERE” clauses

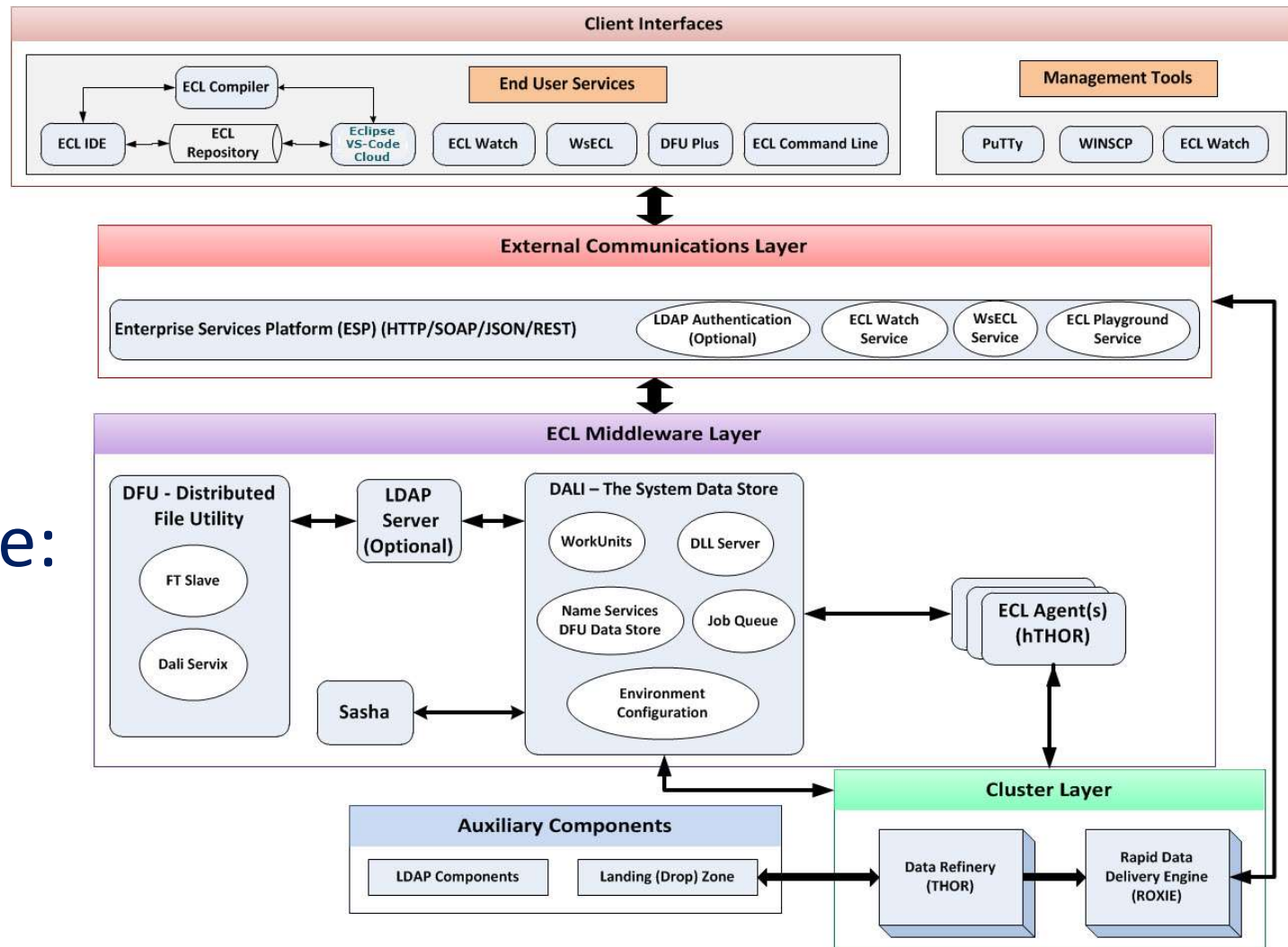
ECL IDE/VS Code and ECL Watch:

- ✓ What is the ECL IDE?
 - ✓ ECL Programmer's Development Environment
 - ✓ ECL Definition File Editing, Syntax Checking
 - ✓ Repository Access and Version Control Management Support
 - ✓ Interactive Query Execution / Result Viewing
- ✓ What is ECL Watch?
 - ✓ Web-based Query Execution Monitoring Tool
 - ✓ Monitor executing Workunits
 - ✓ File Management Tool
 - ✓ File Spray/Despray

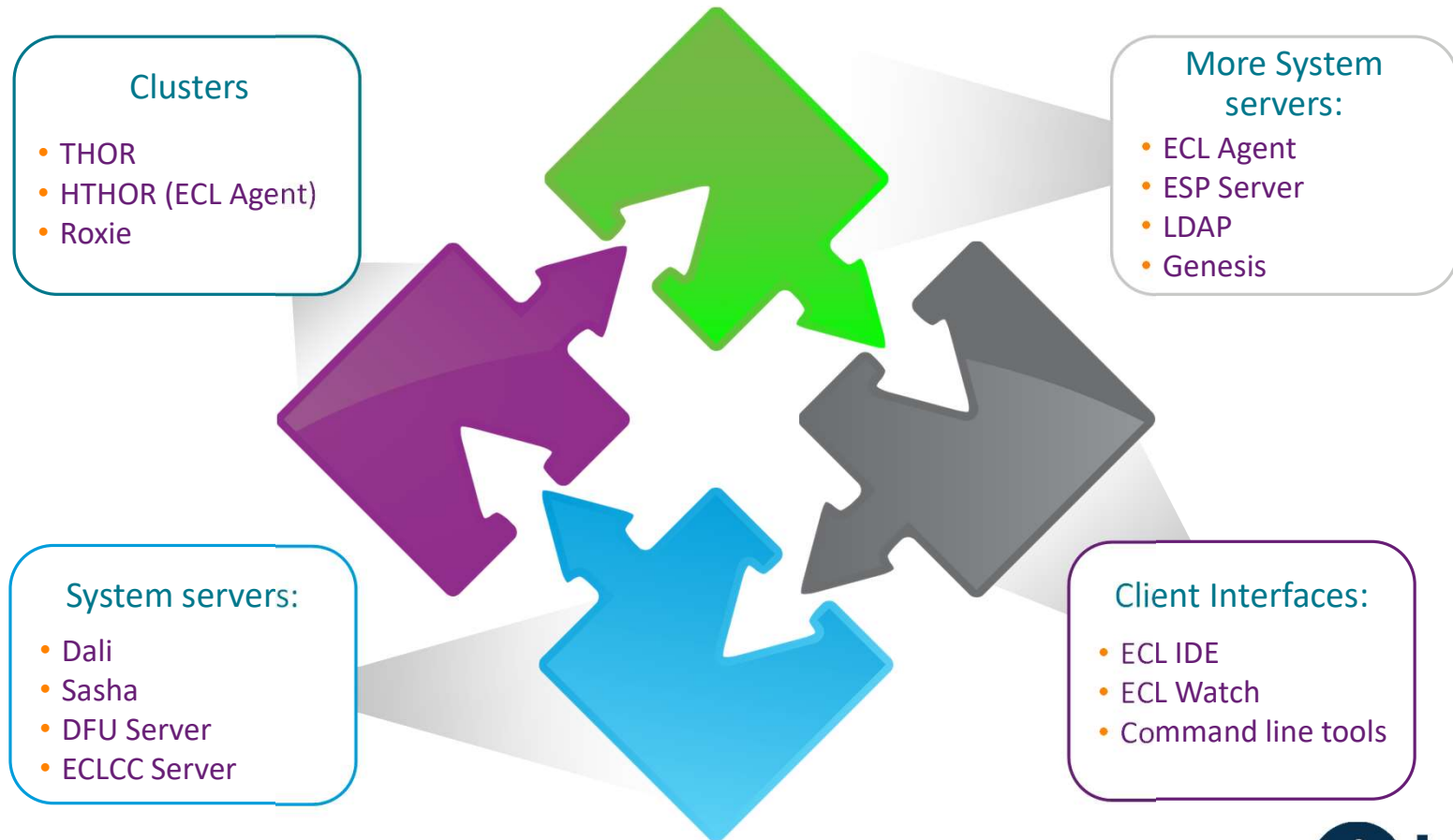
What is ECL and common ECL terms:

- ✓ ECL is a Query and ETL (Extract, Transform, & Load) Language
 - ✓ Scalable, Extensible, Declarative, Non-Procedural
 - ✓ Supports Rapid Data Delivery Engine Queries and Data Refinery ETL operations (ROXIE and THOR)
- ✓ ECL code is made up of ECL Definitions and Actions
 - ✓ ECL Definitions define *what* things are, not *how* to do them
 - ✓ Their order of appearance in source code does not define execution order (ECL is non-procedural)
- ✓ ECL Actions cause compilation and execution
- ✓ ECL Functions are used in ECL expressions
- ✓ ECL files are stored in folders in the ECL Repository

HPCC Architecture:



Components of the HPCC



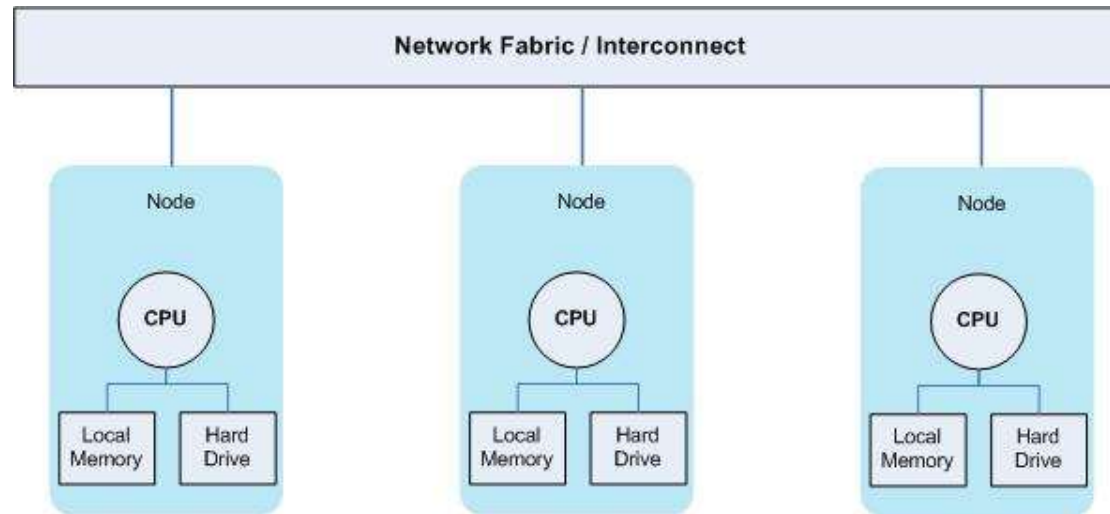
HPCC Clusters

LN's HPCC environment contains clusters which you define and use according to your needs.

There are **two types of clusters** in the HPCC:

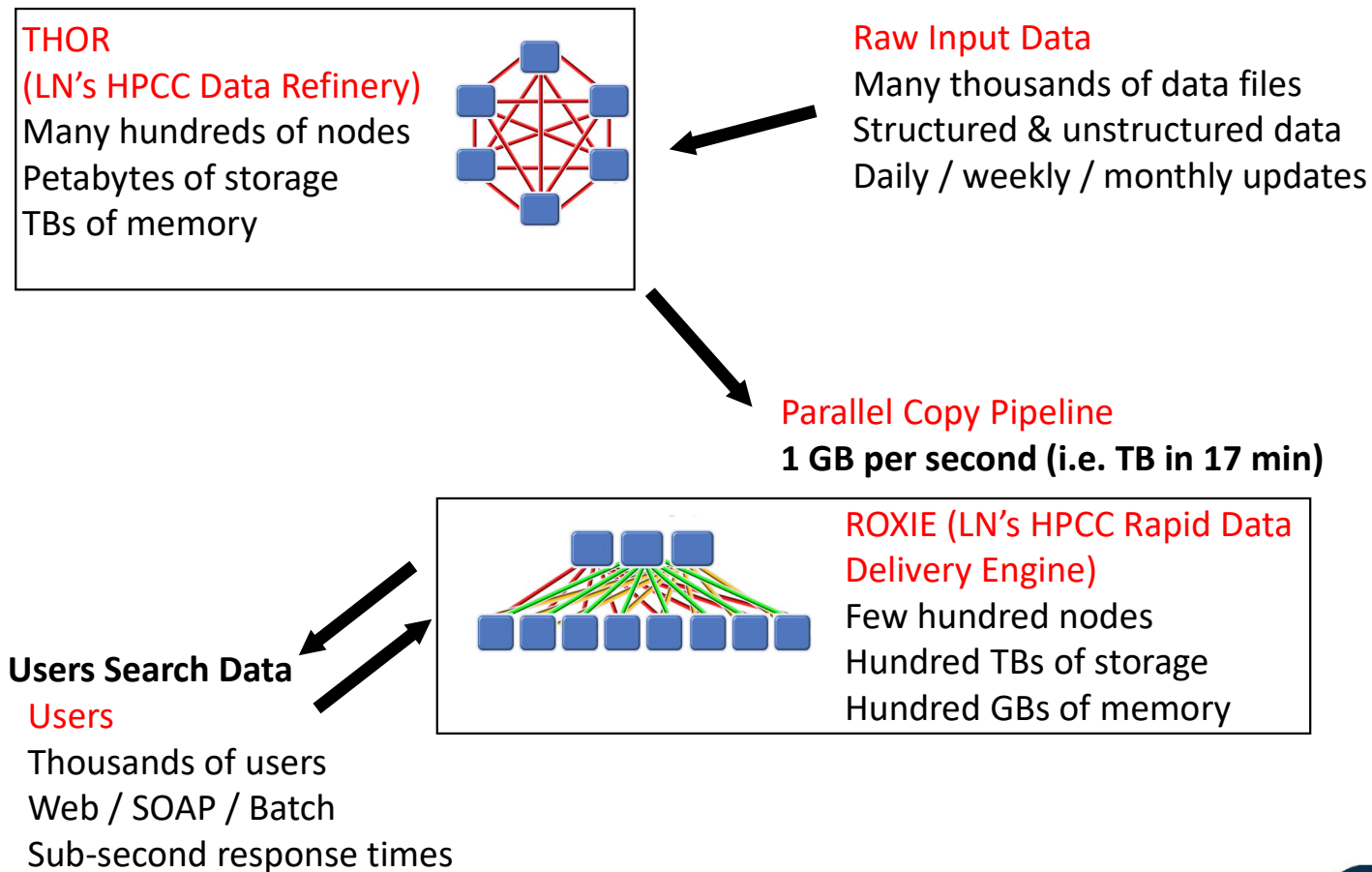
1. Data Refinery (**THOR**) – Used to process every one of billions of records in order to create billions of "improved" records.
The ECL Agent (HTHOR) is also used to process simple jobs that would be an inefficient use of the THOR cluster.
2. Rapid Data Delivery Engine (**ROXIE**) – Used to search quickly for a particular record or set of records.

HPCC Hardware



- Cluster of commercial off-the-shelf components (COTS). Components are homogeneous (all processing/disk storage components same) and the system is tightly coupled. Although similar to a Grid system, nodes are managed *en masse* instead of individually allowing coordinated processing like global sorts.

The Big Picture



Data on the HPCC

THOR/ECL Agent (HTHOR):

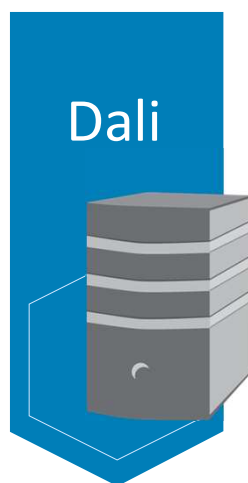
- Data loading is controlled through the Distributed File Utility (DFU) Server.
- Data typically arrives on the Landing Zone (for example, by FTP).
- File movement is initiated by DFU.
- Data is copied from the Landing Zone and is distributed (sprayed) into the Data Refinery (THOR).
- Data can now be further processed (Extract Transform and Load process).
- Data retrieval process places the file back on the landing zone (despraying).
- A single physical file is distributed into multiple physical files across the nodes of a cluster. The aggregate of the physical files creates one logical file that is addressed by the programmers of ECL code.

ROXIE

- Data and queries are compiled in ECL IDE and deployed (published) in ECL Watch.
- Data moves in parallel from THOR nodes to the receiving ROXIE nodes. Parallel bandwidth utilization improves the speed of putting new data into play.

HPCC Middleware

Dali is also known as the System Data Store.



It manages:

- Workunit records
- Logical file directory
- Shared object services

It is used to:

- Configure the environment
- Maintain the message queues that drive job execution and scheduling
- Enforce the LDAP security restrictions for data file scopes and workunit scopes

HPCC Middleware

The **Sasha** server is a companion “housekeeping” server to the Dali server.



It works independently of all other components:

- Allowing a restart to happen at any time without interfering with jobs in flight.

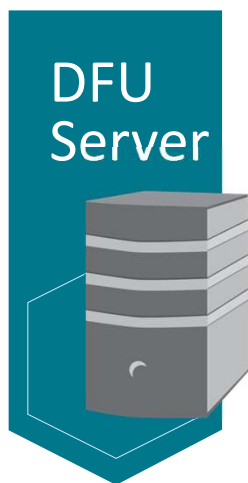
Sasha’s main function is to reduce the stress on the Dali server whenever possible.

Sasha:

- Archives workunits (including DFU Workunits) which:
 - Are stored in a series of folders.
 - Can be restored.
 - Can be manually moved to an alternate or off-site location.
- Reduces the resource utilization on Dali.
- Removes cached workunits and DFU recovery files.

HPCC Middleware

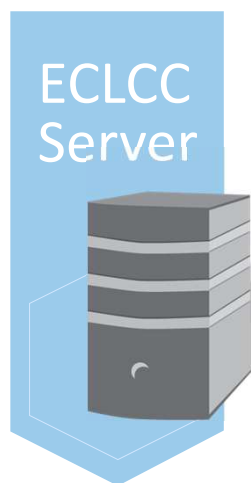
DFU server controls the spraying and despraying operations used to move data onto and out of THOR.



DFU services are available from:

- ECL IDE, using service libraries in ECL code.
- ECL Watch.
- DFU command line interface.

HPCC Middleware



ECLCC Server is the code generator and compiler that translates ECL code.

When workunits are submitted for execution on THOR, they are first converted to executable code by ECLCC Server.

For ROXIE, this process is done at deploy time so that a query can be compiled once, but then executed multiple times.

ECLCC Server also:

- Is used when the ECL IDE requests a syntax check.
- Uses a queue in order to convert workunits one at a time.
 - However, there may be multiple ECLCC Servers deployed in the system to increase throughput and they will automatically load-balance as required.

HPCC Middleware



ECL Agent (HTHOR) is a single node process for executing simple ECL Queries.

Using ECL Agent for simple queries:

- Simplifies the process
- May mean they run faster

There are two circumstances where it may be used:

You may choose to use it as your target:

- If you know your query will run at an acceptable level of performance on a single node.
- If THOR is unavailable – but your query must still be simple enough to run on ECL Agent.

The job is automatically sent:

- If the compiler determines that a query is simple enough to be run on ECL Agent.
- Queries that are not simple enough for ECL Agent are sent to an available THOR cluster.

HPCC Middleware



ESP Server (Enterprise Service Platform) is the inter-component communication server.

ESP Server is a framework that allows multiple services to be “plugged in” to provide various types of functionality to client applications via multiple protocols.

Examples of services that are plugged into ESP include:

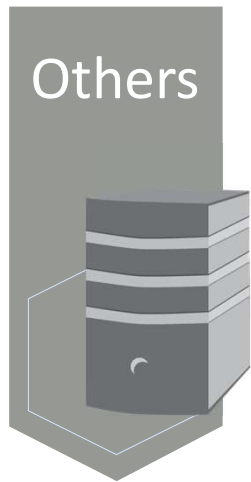
- ECL Playground
Provides an interface to run ECL and verify your HPCC cluster's health.
- WsECL (myWSECL)
Provides an interface to either an ECL definition or a deployed query on a ROXIE cluster.

- ECL Watch
A *web-based* query execution, monitoring and file management interface. It can be accessed via the ECL IDE or a web browser.

Examples of protocols supported by the ESP Server framework include:

- HTTP
- SOAP
- JSON
- REST (REpresentational State Transfer)

HPCC Middleware



LDAP – Lightweight Directory Access Protocol

Works with Dali to enforce the security restrictions for data file and workunit scopes. (Enterprise Only)

Genesis

Operating system on Linux server used to mass update other nodes in the HPCC cluster.

HPCC Client Tools

ECL IDE



A full-featured GUI for ECL development providing access to the ECL repository and many of the ECL Watch capabilities.

Uses various ESP services via SOAP.

Provides the easiest way to create:

- Queries into your data.
- ECL Definitions to build your queries which:
 - Are created by coding an expression that defines how some calculation or record set derivation is to be done.
 - Once defined, can be used in succeeding ECL definitions.

HPCC Client Tools

ECL Watch

A web-based query execution, monitoring and file management interface. It can be accessed via ECL IDE or a web browser.

ECL Watch allows you to:



- See information about active workunits.
- Monitor cluster activity.
- Browse through previously submitted WUs:
 - See a visual representation of the data flow within the WU.
 - Complete with statistics which are updated as the job progresses.
- Search through files and see information including:
 - Record counts and layouts.
 - Sample records.
 - The status of all system servers whether they are in clusters or not.
- View log files.
- Start and stop processes.

HPCC Client Tools

Command line tools:

- DFU PLUS
- ECL

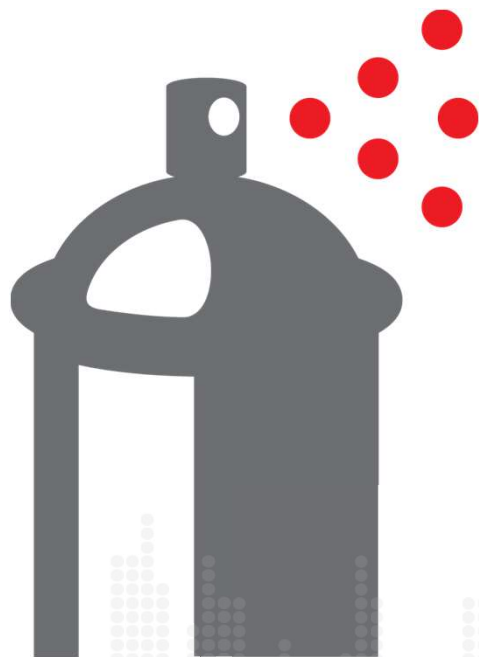


Provide more convenient access to functionality provided by the ECL
Watch web pages.

They work by communicating with the corresponding ESP service via
SOAP.

Customers can write their own client tools using the SOAP interface,
where a SOAP interface has been published.

Spraying data files



- Fixed-length
- Delimited
- XML
- Variable
- BLOB
- JSON

Training Clusters

3 Node THOR

2 Node ROXIE

4 Node THOR

One way ROXIE

<http://play.hpccsystems.com:8010>

<http://training1.hpcc.risk.regn.net:8010>

<http://training2.hpcc.risk.regn.net:8010>

Your Personal Training Clusters:

4 Node THOR

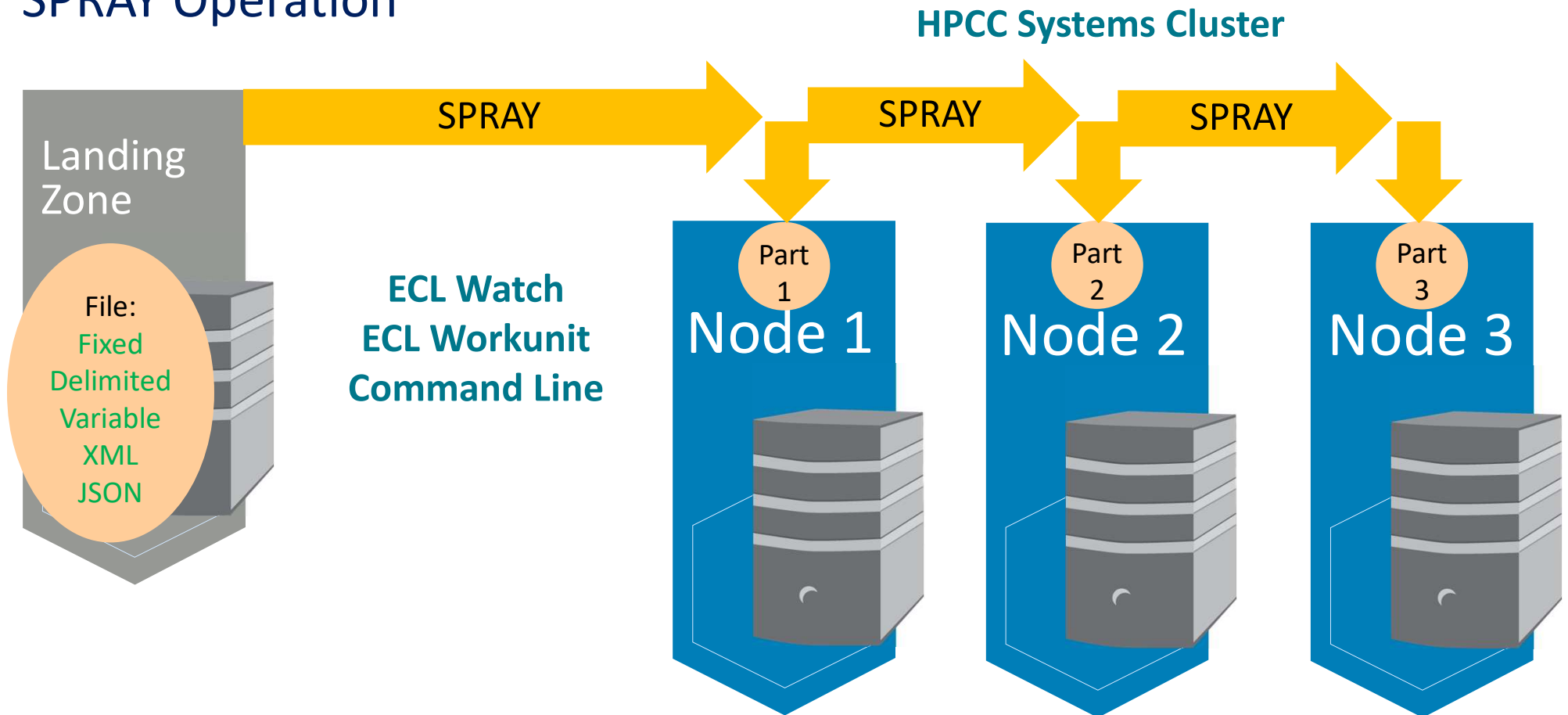
One way ROXIE

<http://training.us-hpccsystems-dev.azure.lnrsg.io:8010/>

(Alternate)

<https://play.hpccsystems.com:18010/>

SPRAY Operation



Referenced in ECL as a single logical file...

Fixed Spray Key Options

- **Target**
 - **Group:** Select the name of THOR cluster to spray to.
 - **Target Scope:** Part of the logical name chosen for the target file(s).
 - **Target Name:** The specific name appended to the Target Scope.
 - **Record Length:** The size of each record.
- **Options**
 - **Overwrite:** Check to overwrite files of the same name.
 - **Compress:** Check to compress the data.
 - **No Split:** Check to spray to a single node
 - **Replicate:** Check to replicate the data
 - **Fail If No Source File** Check to Fail Spray if Source File missing (N/A)

Delimited Spray Options:

- **Target**

- **Group:**

Select the name of THOR cluster to spray to.

- **Target Scope:**

Part of the logical name chosen for the Target Name(s).

- **Target Name:**

The specific name appended to the Name Prefix.

- **Options**

- **Format:**

Select the file format from the drop-list.

- **Max Record Length:**

The length of longest record in the file.

- **Separator:**

The field delimiter in the source file.

- **Omit Separator:**

Check to ignore all separators

- **Escape**

The Escape code used to detect special characters

- **Line Terminator:**

The record delimiter in the source file.

- **Quote:**

The quote character in the source file.

- **Overwrite:**

Check to overwrite files of the same name.

- **Replicate**

Check to replicate the sprayed data

- **No Split**

Check to spray a small file on to a single node

- **Compress:**

Check to compress the data

- **Fail If No Source File**

Check to Fail Spray if Source File missing

- **Record Structure Present**

Check for record field names in first row

- **Quoted Terminator**

Check to indicate that the line terminator is quoted

XML/JSON Spray Options:

- **Target**
 - **Group:** Select the name of THOR cluster to spray to.
 - **Name Prefix:** Part of the logical name that chosen for the file(s).
 - **Target Name:** The specific name appended to the Name Prefix.
 - **Row Tag or Path:** The XML/JSON record delimiter tag.
- **Options**
 - **Format:** Select the file format from the drop-list.
 - **Max Record Length:** The length of longest record in the file.
 - **Overwrite:** Check to overwrite files of the same name.
 - **Compress:** Check to compress the data
 - **Fail If No Source File** Check to Fail Spray if Source File missing
 - **Record Structure Present** Check for record field names in first row

Variable Spray Options:

- **Target**

- **Group:**

Select the name of THOR cluster to spray to.

- **Target Scope:**

Part of the logical name that chosen for the file(s).

- **Target Name:**

The specific name appended to the Target Scope.

- **Options**

- **Source Type**

Select the appropriate source IBM format used to spray:

- *Recfmv* is for IBM RDW files.

Recfmb is for IBM BDW/RDW files.

- *Variable*

Variable-Big Endian

- **Overwrite:**

Check to overwrite files of the same name.

- **Compress:**

Check to compress the data

- **Fail If No Source File**

Check to Fail Spray if Source File missing

- **Record Structure Present**

Check for record field names in first row

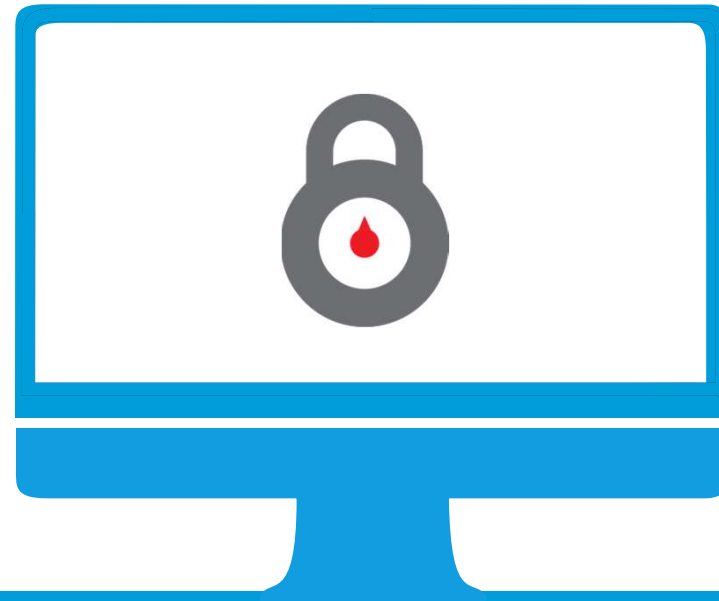
BLOB Spray Options:

- **Target**
 - **Group:** Select the name of THOR cluster to spray to.
 - **Target Name:** The specific name appended to the Name Prefix.
 - **Source Path (Wildcard Enabled):** Part of the logical name that chosen for the file(s).
- **Options**
 - **BLOB Prefix** filename{:length}, filesize{:[B/L] [1 – 8] }
 - **Overwrite:** Check to overwrite files of the same name.
 - **Compress:** Check to compress the data
 - **Fail If No Source File** Check to Fail Spray if Source File missing
 - **Record Structure Present** Check for record field names in first row

Lab Exercises

Spray Lab Exercises:

- 1 and 2



Basic ECL

ECL Definition:

Name := Expression;

- The **Name** must begin with a letter and may only contain letters, numbers, and underscores (_)
- The ECL Definition Operator (:=) is read as:
“is defined as”
- The **Expression** must define either a single value calculation, a Boolean True/False condition, a set of values, or a set of records
- The semi-colon (;) is the required ECL definition terminator

Basic ECL

Case Sensitivity / Formatting / Comments

- ECL is not case-sensitive. White space is ignored allowing formatting for readability
- Block comments are delimited with `/*` and `*/`
- Single line comments begin with `//`
- ECL uses `object.property` syntax to qualify scope and disambiguate references within tables

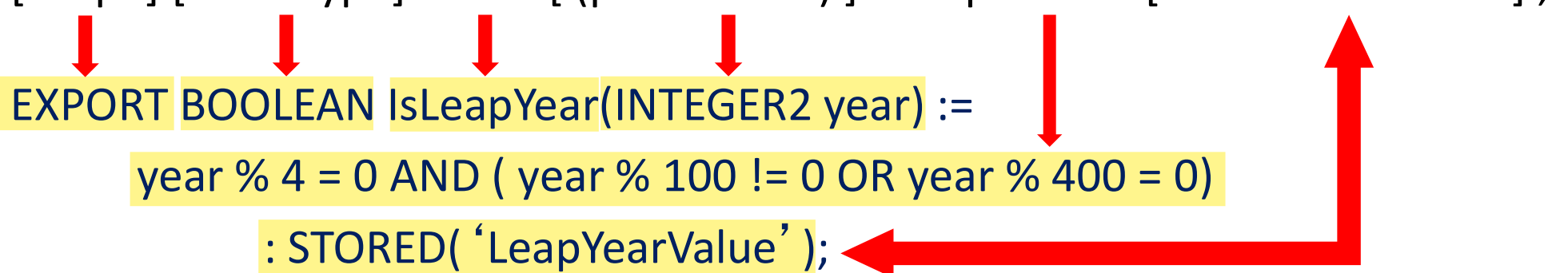
FolderName.Definition `//` reference a definition in another module

Dataset.Field `//` reference a field in a dataset or recordset

Full ECL Definition Syntax

[Scope] [ValueType] Name [(parameters)] := Expression [:WorkflowService] ;

EXPORT BOOLEAN IsLeapYear(INTEGER2 year) :=
year % 4 = 0 AND (year % 100 != 0 OR year % 400 = 0)
: STORED('LeapYearValue');



ECL Definition Types - Boolean

Boolean –

A Boolean definition is a logical expression resulting in a TRUE/FALSE result

IsGoodClass := TRUE;

IsFloridian := People.state = 'FL';

IsSeniorCitizen := People.Age >= 65;

ECL Definition Types - Value

Value –

A Value definition is an arithmetic or string expression with a single-valued result

ValueTrue := 1;

FloridianCount := COUNT(People(IsFloridian));

SeniorAvgAge := AVE(People(IsSeniorCitizen), People.Age);

ECL Definition Types - Set

Set –

A Set definition is a set of explicitly declared constant values or expressions within square brackets. All elements must be the same type.

SetTrueFalseValues := [0, 1];

SetSoutheastStates := ['FL','GA','AL','SC'];

SetStatusCodes := ['1','X','9','W'];

SetInts := [1,2+3,45,def1,7*3,def2];

Set Ordering and Indexing

Declared sets –

must be indexed to access individual elements within the set

```
SetNums := [5,4,3,2,1];
```

```
LastNumInSet := SetNums[5];
```

```
// LastNumInSet contains the value 1
```

Set Ordering and Indexing

Strings (character sets) –

may also be indexed to access individual characters within the string

```
MyString := 'ABCDE';
```

```
MiddleChar := MyString[3]; //MiddleChar contains “C”
```

Substrings –

may be extracted by using 2 periods to separate the beginning and ending element numbers to specify the substring to extract

```
MySubString1 := MyString[2..4]; // 'BCD'
```

```
MySubString2 := MyString[..4]; // 'ABCD'
```

```
MySubString3 := MyString[2..]; // 'BCDE'
```


ECL Definition Types - Recordset

Recordset –

A Recordset definition is a filtered dataset or recordset

FloridaPeople := People(IsFloridian);

SeniorFloridaPeople := FloridaPeople(IsSeniorCitizen);

Recordset Ordering and Indexing

Recordsets –

may also be indexed to access individual records (or ranges) within the recordset or dataset

```
MySortedRecs      := SORT(People(LastName = 'TAYLOR'), FirstName);  
FirstTaylor       := MySortedRecs[1];    //First record in the sorted set  
FirstTaylors      := MySortedRecs[1..3]; //First three records
```

Fields in Records –

may be accessed using indexing to access the individual record then qualifying the field to return

```
FirstTaylorFirstName := MySortedRecs[1].FirstName;  
//First name in first record in the sorted set
```

Datasets, Recordsets, and Filters

Physical **Datasets**:

can be *memory* or *disk* based

Recordsets:

subsets of physical datasets or other recordsets
based on *filtering* conditions

Filters:

are specified in parentheses following the dataset or recordset name in *any* expression:

**Lifestyle_File := Polk.File_Polk_TLS((*INTEGER*)DOB > 191604 AND
(*INTEGER*)DOB <= 196204 AND DOB != ");**

Recordset Filters

- Any Boolean expression within parentheses following a Dataset or Recordset name is a **filter expression**, used to define the specific subset of records to use
- Multiple filter conditions may be specified by separating each filter expression with a comma (,) or explicitly using the AND operator

T_Names := People(Lastname >= 'T', Lastname < 'U');

RateGE7Trds := Trades(trd_rate >= 7);

ValidTrades := Trades(NOT IsMortgage AND NOT IsClosed);

Functions and Parameter Passing

- Any ECL Definition can be defined to accept passed parameters (arguments), making it an **ECL Function**. which does not change the definition's basic type, just makes it more flexible
- Defined parameters always appear in parenthesis following the ECL definition name
- Multiple parameter definitions are separated by commas

DefinitionName([*ValueType*] *AliasName* [=DefaultValue]) := *expression*;

- The required *AliasName* names the parameter passed for use in the *expression*
- The optional *ValueType* defaults to INTEGER if omitted
- The optional *DefaultValue* makes the parameter omittable

Functions and Parameter Passing – Example:

```
Num25 := 25;
```

```
AddFive(INTEGER x=10) := x + 5;
```

```
NumResult1 := AddFive(Num25); // Result is 30
```

```
NumResult2 := AddFive();      // Result is 15
```

```
//Passing a DATASET as a parameter
```

```
MyRec := {STRING1 Letter};
```

```
SomeFile := DATASET([{'A'},{'B'},{'C'},{'D'},{'E'}],MyRec);
```

```
FilteredDS(DATASET(MyRec) ds) := ds(Letter NOT IN ['A','C','E']);
```

```
//passed dataset referenced as “ds” in expression
```

```
OUTPUT(FilteredDS(SomeFile));
```

ECL Definition Visibility (Scoping)

Global (sort of) –

An ECL definition with the EXPORT keyword is available “globally” throughout its own module and in any other module that IMPORTs its module (must be fully qualified)

EXPORT PeopleCount := COUNT(People); //Available “globally”

Module –

An ECL definition with the SHARED keyword is only available throughout its own module

SHARED StateCount := 50; //Available only within its own module

Local –

An ECL definition without EXPORT or SHARED is available only within other EXPORT or SHARED definitions

Num5 := 5; // Local Scope – Available only to NumTotal
EXPORT NumTotal := Num5 + 10 + StateCount;

Qualification

ECL Definition Qualification –

EXPORTed definitions in other modules and **IMPORT**ed by folder names are available for use when qualified by the folder name:

```
IMPORT Companies; // Exported definitions in Companies available  
FloridaCompanies := Companies.File_Company(state='FL');
```

```
IMPORT $; // Exported definitions in current folder available  
FloridaCompanies := $.File_Company(state='FL');
```

```
IMPORT * FROM Companies;  
// All exported definitions in Companies available  
FloridaCompanies := File_Company(state='FL');  
//This usage is not recommended;
```


Qualification (cond)

Data Field Qualification –

Fields within datasets are qualified by prepending the dataset or recordset name

Young := AgeOf(People.dob) < 30; //field in the Person dataset

Implicit Data Field Qualification –

Dataset or Recordset fields come into scope and can be referenced without qualification in filters

YoungPeople := People(dob[1..6] > '197211');

//dob doesn't need to be qualified here because

// it's used to filter the dataset to which it belongs

Actions and Definitions

Actions instigate Workunits, Definitions don't

OUTPUT(People); // will instigate a Workunit (WU)

People; // implicit action, will instigate a Workunit

Actions can be ECL definitions

A := OUTPUT(People); // will NOT instigate a Workunit

Functions that return scalar values can act as Actions

A := COUNT(People); // definition will not instigate a WU

COUNT(People); // WILL instigate a Workunit

Arithmetic Operators:

Division		/
Integer Division		DIV
Modulus Division		%
Multiplication		*
Addition		+
Subtraction		-

Note: Any division by zero (0) defaults to zero (0).

Bitwise Operators:

Bitwise AND		&
Bitwise OR		
Bitwise Exclusive OR		^
Bitwise NOT		BNOT
Bitshift Right		>>
Bitshift Left		<<

Comparison Operators

Equivalence	=
Not Equal	<>
Not Equal	!=
Less Than	<
Greater Than	>
Less Than or Equal	<=
Greater Than or Equal	>=
Equivalence Comparison	<=> returns -1, 0, or 1

More Operators

Recordset Operators

Append +

CompCombined1 := Comp1 + Comp2 + Comp3;

Append & (maintains record order)

CompCombined2 := Comp1 & Comp2 & Comp3;

Set Operators

Append +

SetCombined := [1,2,3] + [4,5,6] + [7,8,9];

Logical Operators

NOT Boolean NOT operation

~(tilde) Boolean NOT operation

AND Boolean AND operation

OR Boolean OR operation

Operators and Expressions:

String Operators

Concatenation +

FullName := TRIM(FirstName) + ' ' + LastName;

Expression Evaluation

- Expressions are evaluated left-to-right and from the inside out (in nested functions). Parentheses may be used to alter the default evaluation order of precedence for all operators.
- Operator evaluation order of precedence is as listed in the *ECL Language Reference*

Expressions: IN Operator

IN Operator

value IN *value_set*

- *value* – The value definition or constant to search for in the *value_set*.
- *value_set* – The set to search.

The **IN** operator is a shorthand for a collection of OR conditions. It searches the *value_set* to find a match for the *value* and returns a Boolean TRUE or FALSE.

```
SetSoutheastStates := ['FL','GA','AL','SC'];
```

```
BOOLEAN IsSoutheastState(String2 state) := state IN SetSoutheastStates;
```


Expressions: BETWEEN Operator

BETWEEN Operator

seekval BETWEEN loval AND hival

- *seekval* – The value to find.
- *loval* – The low value in the inclusive range.
- *hival* – The high value in the inclusive range.

The **BETWEEN** operator is shorthand for an inclusive range check using standard comparison operators (*SeekVal* >= *LoVal* AND *SeekVal* <= *HiVal*). It may be combined with NOT to reverse the logic.

IsOne2Ten(val) := val BETWEEN 1 AND 10;

// is the passed val in the inclusive range of 1 to 10?

Value Types - BOOLEAN

- A Boolean true/false expression
- TRUE and FALSE are reserved ECL keywords and Boolean constants that may be compared to a Boolean type

```
BOOLEAN NBEQ(String L, String R) :=  
    L <> " AND R <> " AND L=R;
```

```
BOOLEAN IsLeapYear(Integer2 year) :=  
    year % 4 = 0 AND  
    ( year % 100 != 0 OR year % 400 = 0);
```

Value Types – SIGNED/UNSIGNED INTEGERS

[IntType] **[UNSIGNED] INTEGER***[n]*

[IntType] **UNSIGNED***n*

- An *n*-byte integer value
- Valid *n* values are 1 to 8
- *IntType* can be BIG_ENDIAN or LITTLE_ENDIAN (LITTLE_ENDIAN is the default)
- If the UNSIGNED keyword is specified the value is always positive, otherwise it may be negative
- INTEGER8 is the default if no size *n* is specified

UNSIGNED1 ctr := 0; // 0 – 255

INTEGER1 ictr := -100; // -128 to 127

**INTEGER BigNum := 9223372036854775807;
 // 9,223,372,036,854,775,807**

Value Types - REAL

REAL[*n*]

- An *n*-byte standard IEEE floating point value. Valid *n* values are 4 (up to 7 significant digits) and 8 (up to 15 significant digits)
- REAL8 is the default if no size *n* is specified

REAL4 PI := 3.14159;

REAL8 TotalBudget := 153347820.75;

REAL r2 := 1.3e-2; //0.013

Value Types – DECIMAL/UDECIMAL

[UNSIGNED] DECIMAL n [_y]

UDECIMAL n [_y]

- A packed decimal value of n total digits (to a maximum of 32) -- if the $_y$ value is present, the y defines the number of decimal places in the value
- If the UNSIGNED keyword is omitted, the rightmost nibble holds the sign
- Using exclusively DECIMAL values in computations invokes BCD math libraries (base-10 math), allowing up to 32 digits of precision (which may be on either side of the decimal point)

DECIMAL7_2 Salary := 75000.00;

DECIMAL6_5 NegativeLeaseFactor := -0.23544;

Value Types - STRING

[StringType] **STRING***[n]*

- A character string of *n* bytes, space padded (not null terminated)
- If *n* is omitted, the string is variable length to the size needed to contain the result of a cast or passed parameter
- Optional *StringType* may be ASCII or EBCDIC (the default is ASCII)

STRING1 Gender := 'M';

EBCDIC **STRING20** CityName := 'BOCA RATON';

Value Types - QSTRING

QSTRING[*n*]

- A variation of STRING using only 6-bits of storage per character, reducing storage requirements for large strings
- Character set is limited to the capital letters A-Z, 0-9, spaces, and the following special characters:
! " # \$ % & ' () * + , - . / ; < = > ? @ [\] ^ _
- If *n* is omitted, the QSTRING is variable length to the size needed to contain the result of a cast or passed parameter

QSTRING15 `CompanyName := 'LEXISNEXIS, INC';`
// uses 12 bytes of storage instead of 15

Value Types - UNICODE

UNICODE $[n]$

- A UTF-16 encoded character string
- Space padded
- If n is omitted, the UNICODE is variable length to the size needed to contain the result of a cast or passed parameter

UNICODE20 `CompanyName := U' LEXISNEXIS';`

Value Types - DATA

DATA[n]

- A packed hexadecimal character string of n bytes, zero padded (not space padded)

DATA9 CID := x'1a2d33548704662988';

Value Types - VARSTRING

VARSTRING[*n*]

- A null-terminated character string containing *n* bytes of data
- If *n* is omitted, the VARSTRING is variable length to the size needed to contain the result of a cast or passed parameter

VARSTRING10 city := 'BOCA RATON';
// uses 11 bytes of storage

Value Types - VARUNICODE

VARUNICODE $[n]$

- A UTF-16 encoded character string
- Null terminated
- If n is omitted, the VARUNICODE is variable length to the size needed to contain the result of a cast or passed parameter

VARUNICODE20 `CompanyName := U' LEXISNEXIS';`

ECL Constants

- String – text enclosed by single quotation marks ' ' (`\'` to include a single quote, `\\` to include a backslash)
STRING20 MyName := 'Richard Taylor';
- Hexadecimal String – Leading “X” with hex value in single quotes
DATA2 CRLF := x'0D0A';
- Data String – Leading “D” with text in single quotes
MyData := D'abcd';
- Unicode String – Leading “U” with text in single quotes
MyUniString := U'abcd';
- VARSTRING – Leading “V” with text in single quotes
MyVarString := V'abcd';
- QSTRING – Leading “Q” with text in single quotes
MyQString := Q'ABCD';

Numeric Constants

Numeric —

numeric constants containing a decimal portion are treated as REAL values and those without are treated as INTEGER. Integer constants may be decimal, hexadecimal or binary values.

MyInt := 10; // value is INTEGER 10

MyInt := 0x0A; // value is INTEGER 10

MyInt := 0Ax; // value is INTEGER 10

MyInt := 0b1010; // value is INTEGER 10

MyInt := 1010b; // value is INTEGER 10

MyReal := 10.0; // value is REAL 10.0

MyReal2 := 0.1e2; // value is REAL 10.0

MyReal3 := 1000.0e-2; // value is REAL 10.0

Value Types - SET

SET [OF *type*]

- *type* – The value type of the set (INTEGER, REAL, BOOLEAN, STRING, UNICODE, DATA, or DATASET(recstruct)). Defaults to INTEGER, if omitted.

SET and **SET OF** define the name or passed parameter as a set of values. The keyword ALL may be used as the passed parameter default value to indicate all possible values for the set, or empty square brackets to indicate no possible value.

SET OF INTEGER1 SetCloseCodes := [65,66,90,114,115,123];

HasCode(INTEGER code, **SET** s) := code IN s;

ClosedAccounts := Accounts(HasCode(acct_status,SetCloseCodes));

TYPEOF

TYPEOF(*expression*)

- *expression* – An expression defining the value type. This may be the name of a data field, passed parameter, function, or definition providing the record type including RECORD structures.

The **TYPEOF** declaration allows you to define a name or parameter whose value type is the same as the passed *expression*. Typically used as the result type of a TRANSFORM function.

```
INTEGER4 Num := 9765;  
TYPEOF(Num) Num1 := 100567;  
TYPEOF(Num) Num2 := 101678;  
TYPEOF(Num) Num100 := 102901;
```

ENUM

ENUM([*type* ,] *name* [=*value*] [, *name* [=*value*] ...])

- *type* – The numeric type of the *values* (defaults to UNSIGNED4).
- *name* – The label for the enumerated *value*.
- *value* – The numeric value to associate with the *name*. If omitted, each *value* is the previous *value* plus one.

The **ENUM** type specifies constant values to make code more readable.

Gender := ENUM(UNSIGNED1, Male, Female, Neutral, Unknown);

MalePersons := Persons(Gender = Gender.Male);

Type Casting

Explicit Casting –

place the value type in parentheses in the expression immediately preceding the element to cast.

STRING10 Value := '34658';

Cnt := (**INTEGER4**) Value;

StrCnt := (**STRING5**) Cnt;

Implicit Casting –

During expression evaluation, different value types may be implicitly cast in order to evaluate the expression, promoting one value type to another.

STRING to INTEGER, INTEGER to REAL, etc.

Type Transfer –

treats the bit pattern as different type

Full Casting Rules –

see the *ECL Language Reference*

RECORD Structure

```
DefinitionName := RECORD  
  fields;  
END;
```

- *DefinitionName* – The name of the RECORD structure.
- *fields* – The data type and name of each data field.

A **RECORD** structure defines the field layout of a DATASET, recordset, INDEX, or TABLE. The keywords RECORD and END may be replaced with the open and close curly braces ({ }) and the semi-colon field delimiters may be replaced with commas.

RECORD Structure Example:

```
EXPORT Layout_Company := RECORD
  UNSIGNED  sic_code;
  STRING1   source;
  STRING120 company_name;
  STRING10  prim_range;
  STRING2   predir;
  STRING28  prim_name;
  STRING4   addr_suffix;
  STRING2   postdir;
  STRING5   unit_desig;
  STRING8   sec_range;
  STRING25  city;
  STRING2   state;
  STRING5   zip;
  STRING4   zip4;
  STRING10  phone;
END;
```

RECORDOF

RECORDOF(*recordset*)

- *recordset* – The set of data records whose RECORD structure to use. This may be a DATASET or any derived recordset.

The **RECORDOF** declaration specifies inheriting just the record layout (without default values) of the specified *recordset*.

```
t := TABLE(People,{LastName,FirstName});
```

```
r := RECORD  
RECORDOF(t);  
UNSIGNED1 NewByte;  
END;
```

DATASET

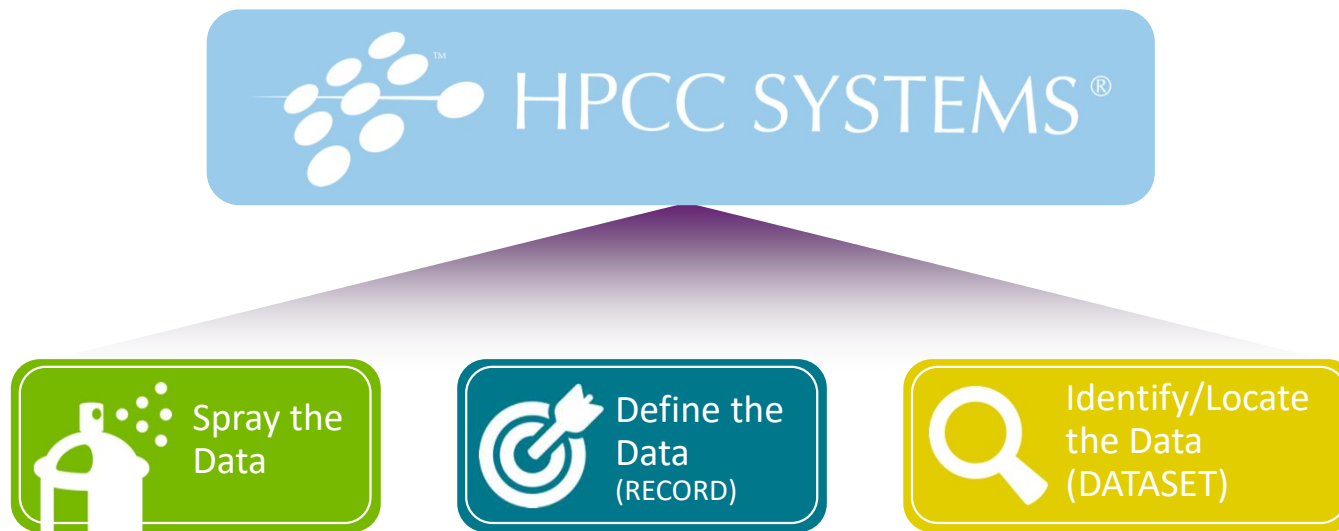
```
name := DATASET( file, recorddef, THOR [ options ] );  
name := DATASET( file, recorddef, CSV [ ( options ) ] );  
name := DATASET( file, recorddef, XML( path, [ options ] ) );  
name := DATASET( file, recorddef, JSON( path, [ options ] ) );  
name := DATASET( file, recorddef, PIPE( command ) );
```

- ✓ *name* – The definition name by which the file is subsequently referenced.
- ✓ *file* – A string constant containing the logical filename.
- ✓ *recorddef* – The RECORD structure of the dataset.
- ✓ *options* – options specific to the dataset type.
- ✓ *path* - A string constant containing the full XPATH to the tag that delimits the records in the *file*
- ✓ *command* – third-party program that creates the dataset.

DATASET introduces a new data file into the system with the specified *recorddef* layout.

Three ECL Data Rules

Before you begin to work on any data in the HPCC cluster, you must always do three things:



RECORD and DATASET example

Layout_Company := **RECORD**

```
UNSIGNED    sic_code;  
STRING120   company_name;  
STRING10    prim_range;  
STRING2     predir;  
STRING28    prim_name;  
STRING4     addr_suffix;  
STRING2     postdir;  
STRING5     unit_desig;  
STRING8     sec_range;  
STRING25    city;  
STRING2     state;  
STRING5     zip;  
STRING4     zip4;  
END;
```

EXPORT File_Company_List := **DATASET**('~CLASS::Company_List', Layout_Company, THOR);

INLINE DATASET

name := **DATASET**(*recordset*, *recorddef*)

- *name* – The definition name by which the file is subsequently referenced.
- *recordset* – A set of data records contained within square brackets (indicating a set definition). Within the square brackets, each record is delimited by curly braces ({}) and separated by commas. The fields within each record are comma delimited.
- *recorddef* – The RECORD structure of the dataset.

DATASET introduces a new table into the system with the specified *recorddef* layout. This form allows you to treat an inline set of data as a data file.

```
NamesRec := RECORD
```

```
  STRING20 first_name;
```

```
  STRING20 last_name;
```

```
END;
```

```
Names := DATASET([{'John','Jones'}, {'Jane','Smith'}], NamesRec);
```


INDEX

name := **INDEX**(*basedataset*, *indexdef*, *indexfile*)

- *name* – The definition name by which the file is subsequently referenced.
- *basedataset* – The recordset for which the index file was created.
- *indexdef* – The RECORD structure of the *indexfile*. Contains key and file position information for referencing into the *baserecordset*. Field names and types must match *baserecordset* field names.
- *indexfile* – A string constant containing the logical filename of the index file created by BUILDINDEX.

INDEX defines an existing index file for use as an index into the *basedataset* (similar to DATASET).

```
EXPORT Key_Vehicle_City := INDEX( File_Vehicles_Plus,  
                                {st,city,lname,fname,filepos},  
                                '~key::vehicle.st.city.fname.lname');
```

Scope and Logical Filenames

- Filenames always start with a scope, followed by the directory structure, and terminating with the name of the file itself. The DFU looks for a file whose name starts with the default scope, followed by the supplied filename:

'DIR1::DIR2::FileName ' **//given this, DFU looks for:**
'SCOPE::DIR1::DIR2::FileName ' **//this file**

- Leading tilde (~) indicates an overridden default scope
'~SCOPE::DIR1::DIR2::FileName' **//given this, DFU looks for:**
'SCOPE::DIR1::DIR2::FileName' **//this file**

ECL Repository Folders

- In the ECL Repository, ECL definition files are stored in Folders
- Folder names must be valid definition names
- ECL definitions that reference other definitions in a different folder must include an IMPORT statement with that folder name
- ECL definitions within a folder that can be referenced from other folders must begin with the EXPORT keyword
- References to definitions in another folder use *foldername.definitionname* syntax



Reserved Keywords

IMPORT folderlist

- folderlist – A comma-delimited list of folder names. This may contain the keyword AS followed by an alias to rename a specific folder in the list.

The **IMPORT** keyword defines a list of folders whose exported definition files are available for use in the current definition text control.

IMPORT \$; //Alias current folder name with \$

IMPORT * FROM TrainingYourName; //override qualification

IMPORT Companies, STD;

IMPORT YellowPages AS YP;

Reserved Keywords (cond)

EXPORT definition

- *definition* – A definition file in a repository folder.

The **EXPORT** keyword explicitly allows other folders to import the specified *definition* file.

```
EXPORT File_Company := DATASET('Company', Layout_Company,FLAT);
```

Special Structures – MODULE

MODULE Structure

```
modulename [ ( parameterlist ) ] := MODULE  
    definitions;  
END;
```

- *modulename* – The ECL definition name of the module.
- *parameterlist* – The parameters available to all *definitions*.
- *definitions* – The ECL definitions that comprise the module. These definitions may receive parameters, and may include actions (such as OUTPUT).

The **MODULE** structure allows you to pass parameters to a set of related ECL definitions. This is similar to the FUNCTION structure except that there is no RETURN and the *definitions* may be EXPORT or SHARED, making them visible outside the **MODULE** structure.

Special Structures – MODULE example

```
EXPORT filterDataset(STRING search, BOOLEAN onlyOldies) := MODULE  
  f := namesTable;  
  SHARED g := IF (onlyOldies, f(age >= 65), f);  
  EXPORT included := g(surname = search);  
  EXPORT excluded := g(surname <> search);  
END;
```

```
//New file -----  
filtered := filterDataset('Halliday', true);  
OUTPUT(filtered.included,,NAMED('Included'));  
OUTPUT(filtered.excluded,,NAMED('Excluded'));
```

Reserved Keywords (cond)

SHARED definition

- *definition* – An ECL definition within a file.

The **SHARED** keyword specifies a definition that is available for use throughout the definition file. Without either SHARED or EXPORT keywords, a definition's scope is limited to the existing text control.

SHARED STRING8 Version := '20021101';

The ECL Prime Directive

- An ECL file must have either:
 - An EXPORT definition (that matches the file name and only one unless MODULE is in play)
 - An Action (implicit or explicit)
- But never together in the same file!
- Sometimes an EXPORT can be interpreted by the Compiler as an Action.

Lab Exercise:

Exercise 3 – Getting Started

- Logging In – Display Options
- Create a Repository Folder
- Creating Queries
- Create an ECL Definition file

Lab Exercise:

Exercise 4 – Define Your Data (Persons)

- MODULE (File_Persons)
- RECORD(Layout) and DATASET(File), both EXPORTed
- Syntax Checking
- Running a Test Query
- Output a Recordset
- Looking at Raw Data

Lab Exercise:

Exercise 5 – Define Your Data (Accounts)

- MODULE (File_Accounts)
- RECORD (Layout) and DATASET (File), both EXPORTed
- Syntax Checking
- Running a Test Query
- Output a Recordset
- Looking at Raw Data

Basic Actions

OUTPUT

[*name* :=] **OUTPUT**(*recordset* [,*format*] [,*file* [,OVERWRITE]])

- *name* – Optional definition name for this action
- *recordset* – The set of records to process
- *format* – The format of the output records: a previously defined RECORD structure, or an "on-the-fly" record layout enclosed in { } braces.
- *file* – Optional name of file to write the records to. If omitted, formatted data stream returns to the command line or ECL IDE program.
- OVERWRITE – Allows file to be overwritten if it exists

The **OUTPUT** action writes the *recordset* to the specified *file* in the specified *format*.

OUTPUT Examples:

OUTPUT(File_Accounts.File); //Equivalent to: File_Accounts.File;

OUTPUT(Persons,{FirstName,LastName}, **NAMED('Names_Only'));**

OUTPUT(MyRecordset,'~CLASS::BMF::NewData', OVERWRITE);

//THOR is default format, but you can also output to:

OUTPUT(MyRecordset,'~CLASS::BMF::NewData', CSV);

OUTPUT(MyRecordset,'~CLASS::BMF::NewData', XML);

OUTPUT(MyRecordset,'~CLASS::BMF::NewData', JSON);

Aggregate Functions - COUNT

COUNT(*recordset*)

COUNT(*valuelist*)

- *recordset* – The set or set of records to process.
- *valuelist* – A comma-delimited list of expressions to count. This may also be a SET of values.

The **COUNT** function returns the number of records in the specified *recordset*.

```
COUNT(Person(per_state IN ['FL','NY']));
```

```
TradeCount := COUNT(Trades);
```

Aggregate Functions - MAX

MAX(*recordset* , *value*)

MAX(*valuelist*)

- *recordset* – The set or set of records to process.
- *value* – The field or expression to find the maximum value of.
- *valuelist* - A comma-delimited list of expressions to find the maximum value of.
This may also be a SET of values

The **MAX** function returns the maximum value of the specified *field* from the specified *recordset*. Returns 0 if the *recordset* is empty.

MaxBal := **MAX**(Trades, Trades.trd_bal);

Aggregate Functions - MIN

MIN(*recordset* , *value*)

MIN(*valuelist*)

- *recordset* – The set or set of records to process.
- *value* – The field or expression to find the minimum value of.
- *valuelist* - *A comma-delimited list of expressions to find the minimum value of. This may also be a SET of values*

The **MIN** function returns the minimum value of the specified *field* from the specified *recordset*. Returns 0 if the *recordset* is empty.

MinBal := **MIN**(Trades, Trades.trd_bal);

Aggregate Functions - SUM

SUM(*recordset* , *value*)

SUM(*valuelist*)

- *recordset* – The set or set of records to process.
- *value* – The expression of field in the *recordset* to sum.
- *valuelist* - A comma-delimited list of expressions to find the sum of. This may also be a SET of values

The **SUM** function returns the additive sum of the value contained in the specified *field* for each record of the *recordset*. Returns 0 if the *recordset* is empty.

SumBal := **SUM**(Trades, Trades.trd_bal);

Aggregate Functions - AVE

AVE(*recordset* , *value*)

AVE(*valuelist*)

- *recordset* – The set or set of records to process.
- *value* – The field or expression to find the average value of.
- *valuelist* - *A comma-delimited list of expressions to find the average value of. This may also be a SET of values*

The **AVE** function returns the average value (arithmetic mean) of the specified *field* from the specified *recordset*. Returns 0 if the *recordset* is empty.

AvgBal := **AVE**(Trades, Trades.trd_bal);

Builder Window Runnable Files (BWR)

- Stored in Repository – Ready to Run!
- Rules:
 - File must have at least one action
 - No EXPORT or SHARED
 - References to other definitions must be fully qualified:

```
IMPORT $;  
Persons := $.File_Persons.File;  
Accounts := $.File_Accounts.File;  
COUNT(Persons);  
COUNT(Accounts);  
OUTPUT(Persons,{ID,LastName,FirstName});
```

Lab Exercise

Exercise 6 – Basic Queries

- Use of IMPORT
- Use of COUNT
- Use of OUTPUT
- Output a Recordset
- Looking at Raw Data

Lab Exercise

Exercise 7a and 7b – Filters

- Basic Filters
- More use of COUNT
- Use of String Indexing
- Use of Logical Operators

ECL Best Practices

- Comparison to other languages
- Atomic Programming
- Growing Solutions
- Ugly ECL
- Easy Optimization

Lab Exercise

Exercise 8a and 8b

- Boolean definitions
- Review naming conventions
- IMPORT and EXPORT needed

SET Function

SET(*recordset*, *field*)

- *recordset* – The set of records from which to derive the SET of values.
- *field* – The field in the *recordset* from which to obtain the values.

The **SET** function returns a SET for use in any set operation (such as the IN operator).

```
r := {STRING1 Letter};  
SomeFile := DATASET([{'A'},{'B'},{'C'},{'D'},{'E'},  
                    {'F'},{'G'},{'H'},{'I'},{'J'}],r);  
SetLettersD_J := SET(SomeFile(Letter > 'C'), Letter);  
y := 'A' IN SetLettersD_J;      //results in FALSE  
z := 'D' IN SetLettersD_J;      //results in TRUE
```

Lab Exercise

Exercise 9

- SET Definitions
- Wrap SET Definitions in a MODULE
- Review Naming Conventions
- IMPORT and EXPORT needed
- Defining SETs
- SET function

Lab Exercise

Exercise 10a-d

- Recordset Definitions
- Reuse of BOOLEAN and SET definitions
- IMPORT and EXPORT needed
- Filtering

Conditional Functions – IF

IF(*expression*, *truevalue*, *falsevalue*)

- *expression* – A conditional expression.
- *truevalue* – If the *expression* is true, this is the value or recordset to return, or an action to perform.
- *falsevalue* – If the *expression* is false, this is the value or recordset to return, or an action to perform.

The **IF** function evaluates the *expression* (which must be a conditional expression with a Boolean result). If the condition evaluates to true, the *truevalue* is evaluated and returned, otherwise the *falsevalue* is evaluated and returned. Both the *truevalue* and *falsevalue* parameters must be of the same type.

```
TradeDate := IF(trades.status IN ['A','D','0','1'], Stringlib.GetDateYYYYMMDD, '');
```

Conditional Functions - MAP

MAP(*condition=>val*,[*condition=>val*,... ,] [*elseval*])

- ✓ *condition* – A conditional expression.
- ✓ *=>* The "results in" operator. Valid only in CASE, MAP and CHOOSESETS.
- ✓ *val* – the value to return if the *condition* is true. This may be a single value, recordset, or action.
- ✓ *elseval* – The value to return if all *conditions* are false. This may be a single value, recordset (optional), or action (optional).

The **MAP** function performs evaluation of the list of *conditions* in sequence. The first *condition* that evaluates as TRUE returns its corresponding *val* as the result of the function. If none of them match, the *elseval* is returned. MAP is an "IF... ELSIF ... ELSIF ... ELSE" type of structure. All *val* and *elseval* expressions must be of the same type.

Recordset Functions

EXISTS(*recordset*)

✓ *recordset* – The set or set of records to process.

The **EXISTS** function returns true if the number of records in the specified *recordset* is > 0 . This is commonly used to detect whether a filter has filtered out all the records.

```
PropertyPeople := People(EXISTS(Property));  
NoPropertyPeople := People(NOT EXISTS(Property));
```

Conditional Functions – MAP Example

```
Valu6012 := MAP(  
    NOT EXISTS(Trades)                => 99,  
    NOT EXISTS(Trades(isValid))       => 98,  
    NOT EXISTS(Trades(isValidDate)) => 96,  
    Count6012);  
/* If there are no trades, Valu6012 gets 99  
   else if there are no valid trades, Valu6012 gets 98  
   else if there are no valid dates, Valu6012 gets 96  
   else, Valu6012 gets Count6012  
*/
```

Conditional Functions – CASE

CASE(*expression*, *caseval* => *val*, [...] *elseval*)

- *expression* – An expression that results in a single value.
- *caseval* – A value to compare against the result of the expression.
- => The "results in" operator.
- *val* – The value to return - a single value, recordset, or action.
- *elseval* – The value to return if the result of the *expression* does not match any of the *caseval* values. This may be a single value, recordset, or action.

The **CASE** function evaluates the *expression* and returns the value whose *caseval* matches the expression result, or the *elseval* if no *caseval* matches. All *val* and *elseval* expressions must be of the same type.

```
NumString123 := CASE(num,1 => 'one',2 => 'two',3 => 'three', 'error');
```


Conditional Functions – CHOOSE

CHOOSE(*expression*, *value*,[..., *value*,] *elseval*)

- *expression* – An arithmetic expression that results in a positive integer and determines which *value* parameter to return.
- *value* – the values to return. There may be as many *value* parameters as necessary to specify all the expected values of the *expression*.
- *elseval* – The value to return when the *expression* returns an out-of-range value. The last parameter is always the *elseval*.

The **CHOOSE** function evaluates the *expression* and returns the *value* parameter whose ordinal position in the list corresponds to the result of the *expression*, or the *elseval* if none matches. All the *value* and *elseval* expressions must be of the same type.

NumString123 := **CHOOSE**(num,'one','two','three', 'error');

Conditional Functions – REJECTED/WHICH

REJECTED(*condition*,...,*condition*)

- *condition* – A conditional expression to evaluate.

The **REJECTED** function returns the ordinal position of the first in the list of *conditions* to return false (zero if all are true).

```
RejectCode := REJECTED(Person.income > 20000,  
                        Person.age < 80,  
                        Person.state <> 'NY');
```

WHICH(*condition*,...,*condition*)

- *condition* – A conditional expression to evaluate.

The **WHICH** function returns the ordinal position of the first in the list of *conditions* to return true (zero if all are false)

```
AcceptCode := WHICH(Person.state IN SetSoutheastStates,  
                    Person.state IN SetMidwestStates,  
                    Person.state IN SetNorthwestStates);
```

Mathematical Functions – ROUNDing Decimals

ROUND(*realvalue*)

- *realvalue* – The floating-point value to round.

The **ROUND** function returns the rounded INTEGER of the *realvalue* by using standard arithmetic rounding (decimal portions less than .5 round down, and decimal portions greater than or equal to .5 round up).

```
Amount := 1000.49;
```

```
RoundAmount := ROUND(Amount); // Rounded amount is 1000;
```

Special Structures – FUNCTION

FUNCTION Structure

```
[resulttype] funcname( parameterlist ) := FUNCTION  
    code;  
    RETURN returnvalue;  
END;
```

- *resulttype* – The return value type of the function. If omitted, the type is implicit from the *returnvalue* expression.
- *funcname* – The ECL definition name of the function.
- *parameterlist* – The parameters to pass to the *code* – available to all definitions in the FUNCTION's *code*.
- *code* – The definitions in the FUNCTION's process.
- **RETURN** – Specifies the function's *returnvalue* expression.
- *returnvalue* – The value, expression, recordset, row (record), or action to return.

The **FUNCTION** structure allows you to pass parameters to a set of related ECL definitions. This makes it possible to pass parameters to other non-exported definitions without the need to parameterize all of those as well.

Without FUNCTION Example:

```
isTradeRateEQSince(STRING1 rate,age) := Trades.trd_rate = rate AND  
                                         ValidDate(trades.trd_drpt) AND  
                                         AgeOf(trades.trd_drpt) <= age;
```

```
isPHRRateEQSince(STRING1 rate,age) := EXISTS(Prev_rate(phr_rate = rate,  
                                                ValidDate(phr_date),  
                                                AgeOf(phr_date) <= age,  
                                                phr_grid_flag = TRUE));
```

```
isAnyRateEQSince(STRING1 rate,age) := isTradeRateEQSince(rate,age)  
OR  
isPHRRateEQSince(rate,age);
```

With FUNCTION Example

```
EXPORT isAnyRateEQSince(STRING1 rate,age) := FUNCTION
```

```
isTradeRateEQSince := Trades.trd_rate = rate AND  
    ValidDate(trades.trd_drpt) AND  
    AgeOf(trades.trd_drpt) <= age;
```

```
isPHRRateEQSince := EXISTS(Prev_rate(phr_rate = rate,  
    ValidDate(phr_date),  
    AgeOf(phr_date) <= age,  
    phr_grid_flag = TRUE));
```

```
RETURN isTradeRateEQSince OR isPHRRateEQSince;  
END;
```

Lab Exercises

Exercise 11 and 12

- Add parameters to a simple ECL Value definition
- Use of FUNCTION structure

More Recordset Functions - SORT

SORT(*recordset*, *value*)

- *recordset* – The set of records to process.
- *value* – An expression or key field in the *recordset* on which to sort. A leading minus sign (-) indicates a descending order sort. You may have multiple *value* parameters to indicate sorts within sorts.

The **SORT** function sorts the *recordset* according to the *values* specified. Any number of *value* parameters may be supplied, with the leftmost being the most significant sort criteria. A leading minus sign (-) on any *value* parameter indicates a descending sort for that one parameter.

```
Business_Contacts_Sort := SORT(Business_Contacts_Dist,  
                                company_name, company_title,  
                                lname, fname, mname, name_suffix,  
                                zip, prim_name, prim_range);
```

```
HighestBals := SORT(ValidBalTrades, -trades.trd_bal);
```

```
UCC_Sort := SORT(UCC_Dist, file_state, orig_filing_num);
```


More Recordset Functions – SORT Example

- Training_Examples.SORT_Example

More Recordset Functions - DEDUP

DEDUP(*recset* [,*condition* [,**ALL**][**BEST** (*sort-list*)],[,**KEEP** *n*] [,*keep*]])

- *recset* – The set of records to process.
- *condition* – The expression that defines “duplicate” records.
- **ALL** –Matches all records to each other using the *condition*, not just adjacent records.
- **BEST**–Provides additional control over which records are retained from a set of "duplicate" records. The first in the <sort-list> order of records are retained. BEST cannot be used with a KEEP parameter greater than 1.
- **KEEP** *n* –Specifies keeping *n* number of duplicates. The default is 1.
- *keep* –LEFT (the default) keeps the first and RIGHT keeps the last.

The **DEDUP** function removes duplicate records from the *recordset*. The *condition* defines what constitutes “duplicate” records.

```
Company_Dedup := DEDUP(Company_Init(zip<>0), company_name,  
                        zip, prim_name, prim_range, sec_range, ALL);
```

LEFT and RIGHT Keywords

LEFT.*field* / RIGHT.*field*

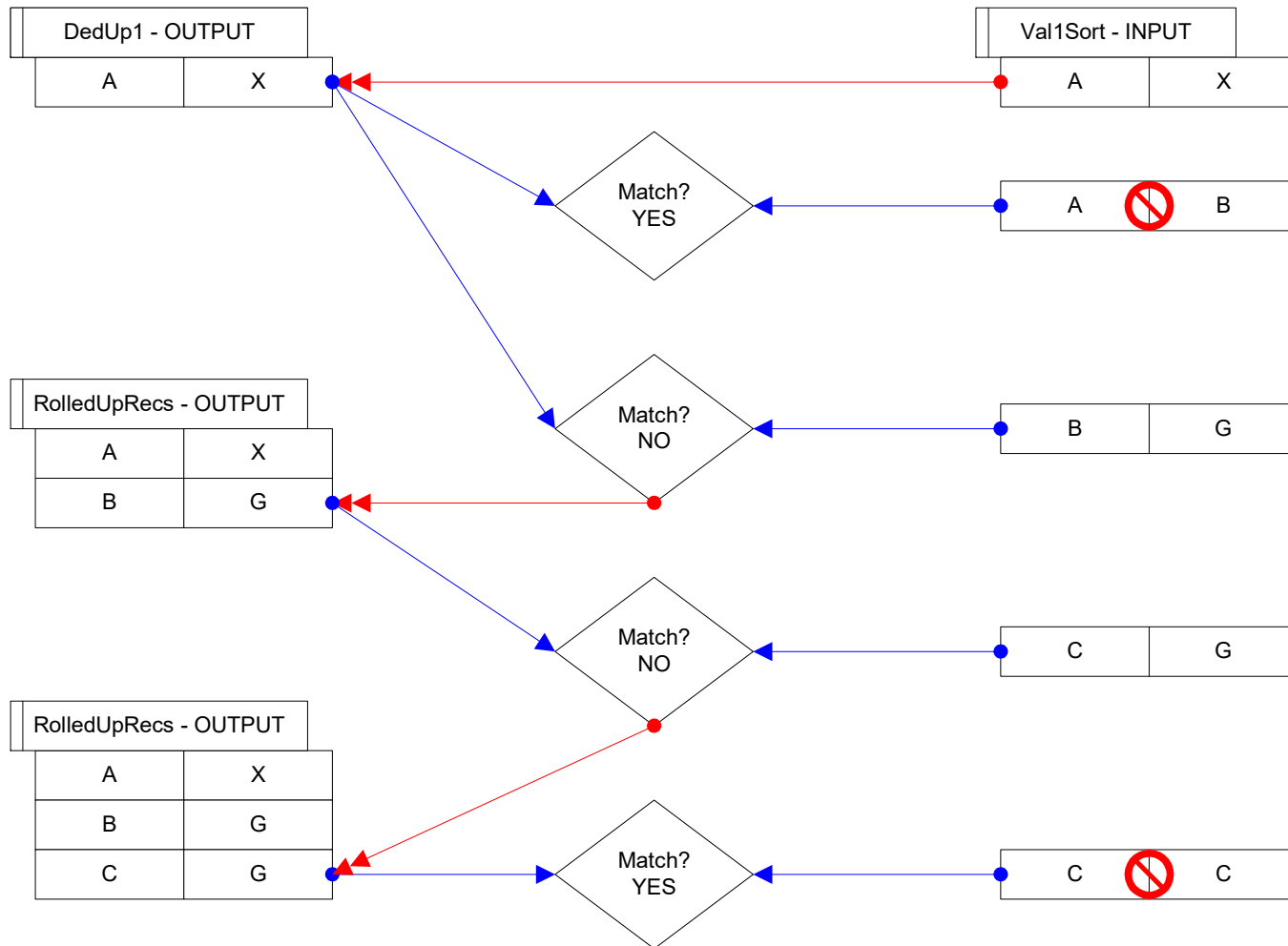
The **LEFT** and **RIGHT** keywords qualify which record a *field* is from in those operations that use pairs of records, such as DEDUP, JOIN, etc. This helps prevent any code ambiguity for the compiler.

```
SortedRecs := SORT(Person,per_last_name,per_first_name);  
DeDupedRecs := DEDUP(SortedRecs,  
    LEFT. per_last_name = RIGHT. per_last_name AND  
    LEFT. per_first_name = RIGHT. per_first_name);
```

More Recordset Functions – DEDUP Example

- Training_Examples.DEDUP_Example

Simple DEDUP Functional Example Diagram



SETs and DATASETs

The elements of any SET can be extracted by applying that SET as the first parameter of any inline dataset:

```
CountSet := [1000,2345,160,6700,99453,2346];
```

```
IsSETUnique(SET OF INTEGER X) := FUNCTION
```

```
  ds1 := DATASET(X, {INTEGER value1});
```

```
  ds2 := DATASET(X, {INTEGER value2});
```

```
  RETURN NOT EXISTS(ds1(COUNT(ds2(ds2.value2 = ds1.value1)) > 1));
```

```
END;
```

```
IsSETUnique(CountSet);
```

Lab Exercise

Exercise 13a – 13c

- Value Definitions
- Building on all ECL Language Statements previously discussed.

DEDUP with ALL

One more before we go!

GROUP Function (used with DEDUP ALL)

GROUP(*recordset* [, *breakcriteria* [, **ALL**]])

- *recordset* – The set of records to fragment.
- *breakcriteria* –An expression that specifies how to fragment the *recordset*. If omitted, the *recordset* is ungrouped.
- **ALL** –Indicates the *breakcriteria* is applied without regard to any previous order. If omitted, GROUP assumes the *recordset* is already sorted in *breakcriteria* order.

The **GROUP** function fragments the *recordset* into a set of sets. This allows aggregations and other operations (such as SORT, DEDUP, ITERATE, and ROLLUP) to occur within defined subsets of the data. Successive GROUP functions against the same *recordset* are implicitly nested.

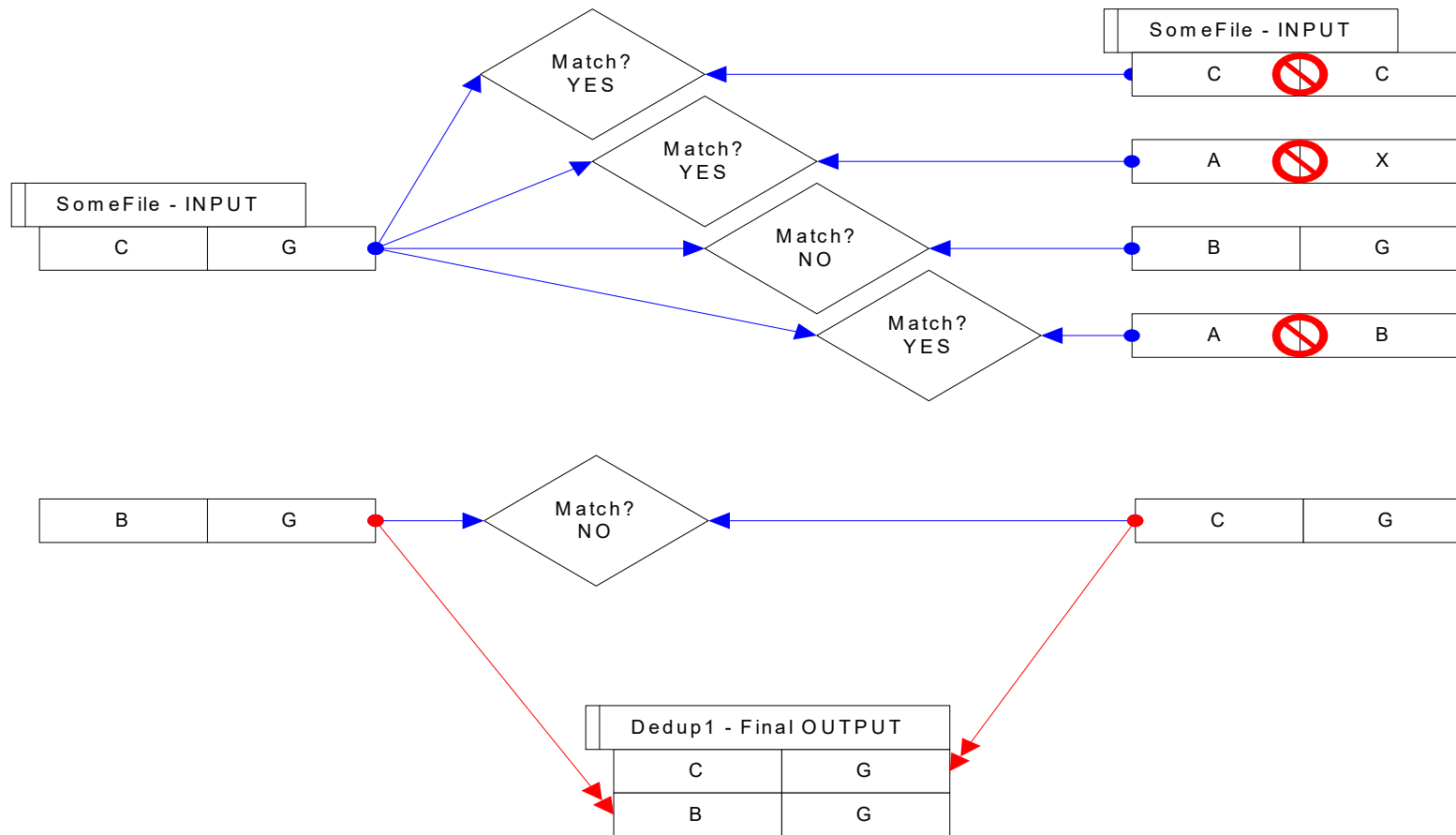
```
UCCSort := SORT(UCC_Dist, file_state, orig_filing_num, LOCAL);  
Grpd    := GROUP(UCCSort, file_state, orig_filing_num, LOCAL);  
Sort_1  := SORT(Grpd, filing_date, filing_type);  
Dedup_1 := DEDUP(Sort_1, (LEFT.filing_date <> RIGHT.filing_date  
                     OR LEFT.filing_type <> RIGHT.filing_type));
```

More Recordset Functions – DEDUP with ALL

- Training_Examples.DEDUP_ALL_Example

DEDUP ALL Functional Example Diagram

LEFT.Value2 IN ['G','C','X'] AND RIGHT.Value2 IN ['X','B','C']



End of Part 1!

More to Come!!

